

Alineamiento Global

Alumno: Josue Samuel Philco Puma

5 de mayo de 2026

1. Introducción

El algoritmo de **Needleman-Wunsch** es el método fundacional de la bioinformática para realizar el alineamiento global de dos secuencias (como ADN o proteínas). Utiliza una estrategia de programación dinámica para comparar ambas cadenas de principio a fin, buscando la organización óptima que maximice las coincidencias y minimice las diferencias. A través de una matriz de puntuación que penaliza huecos (gaps) y premia identidades, este algoritmo garantiza encontrar la mejor relación evolutiva o funcional posible entre dos secuencias de longitud similar.

2. Ejercicios

2.1. Algoritmo Needleman-Wunsch

Ahora vamos a pasar a implementar el algoritmo **Needleman-Wunsch** para el alineamiento global, este algoritmo va a seguir unos pasos similares a lo que era el algoritmo de **Smith-Waterman**, solo que modificando ciertos pasos:

1. Primero vamos a tener que leer dos secuencias a partir de dos archivos **FASTA**, además de los parámetros de puntuación (**match**, **mismatch** y **gap**).
2. Vamos a crear la matriz $(n+1)(m+1)$ donde se va a almacenar los valores óptimos de alineamiento.
3. Se inicializa la primera columna de la matriz asignando penalizaciones acumuladas por gaps, de modo que cada celda $(i, 0)$ toma el valor $i * gap_penalty$. y la primera fila donde cada $(0, j)$ va a tomar el valor $j * gap_penalty$.
4. Para cada celda (i, j) se calcula los valores de la diagonal, superior e izquierdo, a cada celda se le va a asignar el valor máximo entre los valores calculados.
5. Al finalizar la construcción de la matriz se inicia el proceso de **backtracking** comparando el valor actual con los posibles movimientos.
6. Si el movimiento seleccionado es diagonal, se agregan los caracteres correspondientes de ambas secuencias al alineamiento; si es hacia arriba, se agrega el carácter de la primera secuencia y un gap en la segunda; y si es hacia la izquierda, se agrega un gap en la primera secuencia y el carácter de la segunda.
7. Por último se retorna el alineamiento óptimo junto con su puntuación final.

Ahora que tenemos los pasos correspondientes debemos definir los parámetros correspondientes para este algoritmo, los parámetros que usaremos son los siguientes:

```
1 const int MATCH_SCORE = 2;  
2 const int MISMATCH_PENALTY = -1;  
3 const int GAP_PENALTY = -2;
```

Con estos parámetros de puntuación vamos a estar haciendo probando el algoritmo **Needleman-Wunsch**, el algoritmo como tal esta implementando con programación dinámica y sería de esta forma:

```
1 struct AlignmentResult {  
2     string aligned_seq1;  
3     string aligned_seq2;  
4     int score;  
5     int start_pos_seq1, end_pos_seq1;  
6     int start_pos_seq2, end_pos_seq2;  
7     vector<vector<int>> score_matrix;  
8     vector<pair<int, int>> alignment_path;
```

```

9  };
10
11  AlignmentResult needleman_wunsch(const string &sequence1, const string &sequence2,
12      int match_score, int mismatch_penalty, int gap_penalty) {
13      int n = (int)sequence1.size();
14      int m = (int)sequence2.size();
15
16      vector<vector<int>> score_matrix(n + 1, vector<int>(m + 1, 0));
17
18      for (int i = 0; i <= n; i++) {
19          score_matrix[i][0] = i * gap_penalty;
20      }
21      for (int j = 0; j <= m; j++) {
22          score_matrix[0][j] = j * gap_penalty;
23      }
24
25      for (int i = 1; i <= n; i++) {
26          for (int j = 1; j <= m; j++) {
27              int score_diagonal = score_matrix[i - 1][j - 1] + (sequence1[i - 1] ==
28                  sequence2[j - 1] ? match_score : mismatch_penalty);
29              int score_up = score_matrix[i - 1][j] + gap_penalty;
30              int score_left = score_matrix[i][j - 1] + gap_penalty;
31
32              if (score_diagonal >= score_up && score_diagonal >= score_left) {
33                  score_matrix[i][j] = score_diagonal;
34              }
35              else if (score_up >= score_left) {
36                  score_matrix[i][j] = score_up;
37              }
38              else {
39                  score_matrix[i][j] = score_left;
40              }
41          }
42      }
43
44      string aligned_sequence1, aligned_sequence2;
45      int i = n, j = m;
46
47      while (i > 0 && j > 0) {
48          int score_diagonal = score_matrix[i - 1][j - 1] + (sequence1[i - 1] ==
49              sequence2[j - 1] ? match_score : mismatch_penalty);
50          int score_up = score_matrix[i - 1][j] + gap_penalty;
51          int score_left = score_matrix[i][j - 1] + gap_penalty;
52
53          if (score_matrix[i][j] == score_diagonal) {
54              aligned_sequence1 += sequence1[i - 1];
55              aligned_sequence2 += sequence2[j - 1];
56              i--;
57              j--;
58          }
59          else if (score_matrix[i][j] == score_up) {
60              aligned_sequence1 += sequence1[i - 1];
61              aligned_sequence2 += '-';
62              i--;
63          }
64          else {
65              aligned_sequence1 += '-';
66              aligned_sequence2 += sequence2[j - 1];
67              j--;
68          }
69      }
70
71      while (i > 0) {

```

```

69     aligned_sequence1 += sequence1[i - 1];
70     aligned_sequence2 += '-';
71     i--;
72 }
73
74 while (j > 0) {
75     aligned_sequence1 += '-';
76     aligned_sequence2 += sequence2[j - 1];
77     j--;
78 }
79
80 reverse(aligned_sequence1.begin(), aligned_sequence1.end());
81 reverse(aligned_sequence2.begin(), aligned_sequence2.end());
82
83 return {aligned_sequence1, aligned_sequence2, score_matrix[n][m], 0, n - 1, 0,
84         m - 1, score_matrix, {}};

```

Entonces vamos a ejecutar este algoritmo con un total de 4 secuencias en total, 2 secuencias pequeñas y 2 secuencias largas.

Identificador	Secuencia
seq1_human_beta_globin	ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTAC TGCCCTGTGGGGCAAGGTGAACGTGGATG
seq2_mouse_beta_globin	ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTAC TGCCAGTGGGGCAAGGTGAACGTGGATG

Cuadro 1: Secuencias pequeñas

Identificador	Secuencia
NC_012920.1 Homo sapiens	GATCACAGGTCTATCACCCCTATTAACCACTCACGGGAG CTCTCCATGCATTTGGTATTTTCGTCTGGGG...
NC_005089.1 Mus musculus	GTTAATGTAGCTTAATAACAAAGCAAAGCACTGAAAAT GCTTAGATGGATAATTGTATCCCATAAACAC...

Cuadro 2: Secuencias largas

Entonces tendremos los siguientes resultados para estas secuencias.

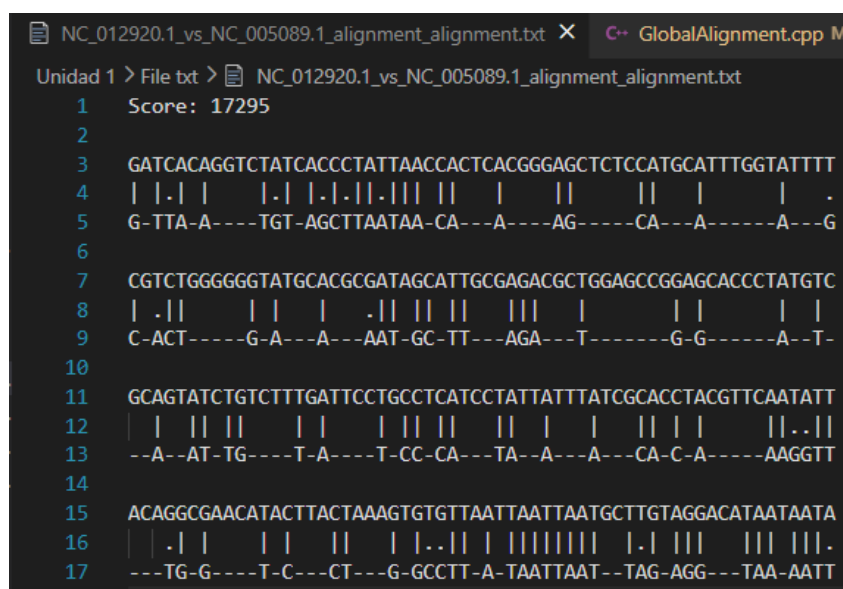
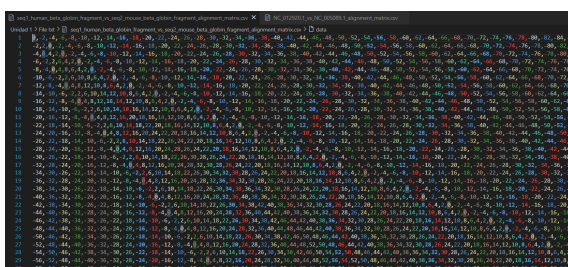


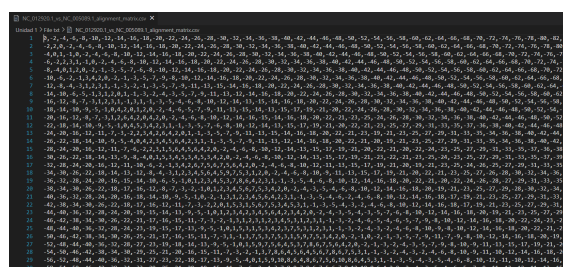
Figura 1: Resultados de la alineación con las secuencias mitocondriales.

```
seq1_human_beta_globin_fragment_vs_seq2_mouse_beta_globin_fragment_alignment_align
Unidad 1 > File txt > seq1_human_beta_globin_fragment_vs_seq2_mouse_beta_globin_fr
1 Score: 131
2
3 ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCCTGTGGGGCAAGGTGAAC
4 |||
5 ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCAGTGGGGCAAGGTGAAC
6
7 GTGGATG
8 |||
9 GTGGATG
```

Figura 2: Resultados de la alineación con las secuencias pequeñas.



(a) Matriz de las secuencias de prueba.



(b) Matriz de las secuencias mitocondriales.

Figura 3: Matriz de las secuencias de prueba (a) y secuencias mitocondriales (b)

3. Comparación entre Needleman-Wunsch y Smith–Waterman

Ahora que tenemos los resultados correspondientes, debemos realizar comparaciones entre el algoritmo **Smith-Waterman** de alineamiento local y el algoritmo **Needleman-Wunsch**, esto lo haremos por los resultados obtenidos y los tiempos de ejecución.

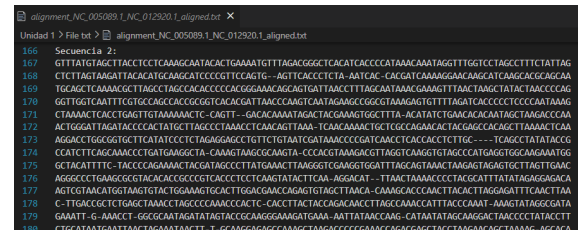
3.1. Diferencia en los resultados

Para el algoritmo **Needleman-Wunsch** como tal es para alinear dos secuencias de extremo a extremo sin excepción, además este algoritmo va a forzar a alinearse introduciendo gaps y pagando una penalización por cada uno. En estas secuencias mitocondriales, la Seq2 recién empieza a parecerse a Seq1 desde la posición 576, entonces los primeros 576 bp divergentes igual se alinearon a la fuerza, bajando el score a 17 295.

Smith-Waterman en cambio no tiene esa obligación. Busca la subregión donde ambas secuencias se parecen más, alinea solo esa parte y descarta los extremos divergentes. Por eso reporta un score mayor (18 533) y un alineamiento más limpio, porque no está siendo penalizado por zonas que biológicamente no tienen homología relevante.

```
alignment_NC_005089.1_NC_012920.1_info.txt
Unidad 1 > File txt > alignment_NC_005089.1_NC_012920.1_info.txt
1 Score: 18533
2 Start/End Seq1: [0 - 15859]
3 Start/End Seq2: [576 - 16568]
4
```

Figura 4: Score con Smith-Waterman.



(b) Secuencia alineada 2.

Figura 5: Secuencias alineadas con Smith-Waterman.

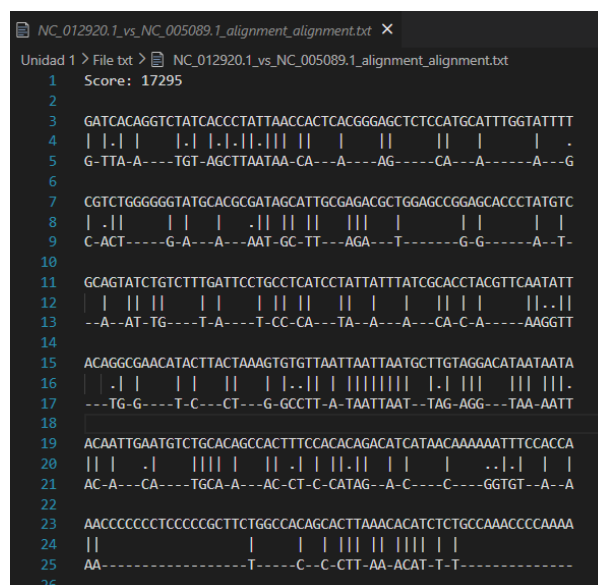


Figura 6: Secuencias alineadas con Needleman-Wunsch

```
PS D:\UNSA EPCC\9no semestre\Bioinformatica\Unidad 1> .\GlobalAlignment.exe .\File
Time: 12.772 seconds
Matrix saved to: File txt\NC_012920.1_vs_NC_005089.1_alignment_matrix.csv
Alignment saved to: File txt\NC_012920.1 vs NC_005089.1 alignment alignment.txt
```

Figura 7: Tiempo de ejecución en Needleman-Wunsch con genomas mitocondriales.

```

PS D:\UNSA EPCC\9no semestre\Bioinformatica\Unidad 1> .\LocalAlignment.exe .\Files\WCBI_musculus.fasta .\Files\WCBI_mitochondrion.fasta
Match score: 2
Mismatch: -1
Gap penalty: -2
Sequences exported to: File txt/sequences1.txt
Sequences exported to: File txt/sequences2.txt
Aligning NC_005089.1 with NC_012920.1...
Time: 38.882 seconds

Alignment matrix exported to: File txt/alignment_NC_005089.1_NC_012920.1_matrix.txt
Alignment path exported to: File txt/alignment_NC_005089.1_NC_012920.1_path.txt
Alignment info exported to: File txt/alignment_NC_005089.1_NC_012920.1_info.txt
Aligned sequences exported to: File txt/alignment_NC_005089.1_NC_012920.1_aligned.txt
PS D:\UNSA EPCC\9no semestre\Bioinformatica\Unidad 1>

```

Figura 8: Tiempo de ejecución en Smith-Waterman con genomas mitocondriales.

4. Visualización del Heatmap

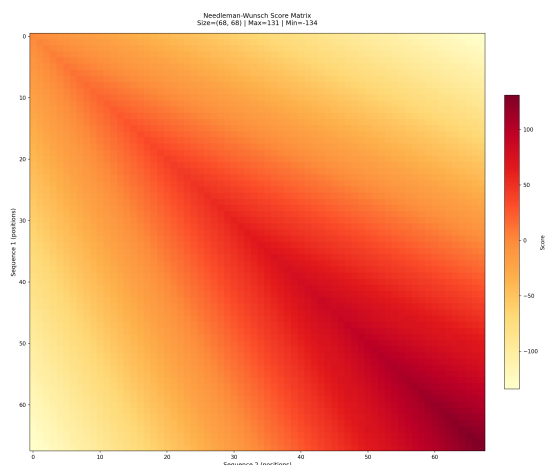
Ahora que tenemos la matriz correspondiente, vamos a utilizarlo para mostrar los Heatmaps correspondientes para las secuencias utilizadas. Esto lo realizaremos con Python.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import sys
4 import os
5
6 def load_matrix(matrix_path):
7     return np.loadtxt(matrix_path, delimiter=",", dtype=int)
8
9 def plot_heatmap(matrix_path, header1, header2):
10     print(f"Reading matrix: {matrix_path}")
11
12     H = load_matrix(matrix_path)
13     print(f"Matrix size: {H.shape}")
14
15     output_img = os.path.join("Heatmaps", header1 + "_vs_" + header2 + "
16                               _alignment_matrix.png")
17
18     fig, ax = plt.subplots(figsize=(14, 12))
19
20     im = ax.imshow(H, cmap="YlOrRd", aspect="auto", interpolation="nearest")
21     plt.colorbar(im, ax=ax, label="Score", fraction=0.03, pad=0.04)
22
23     ax.set_xlabel("Sequence 2 (positions)", fontsize=11)
24     ax.set_ylabel("Sequence 1 (positions)", fontsize=11)
25     ax.set_title(
26         f"Needleman-Wunsch Score Matrix\nSize={H.shape} | Max={H.max()} | Min={H.
27           min()} ",
28         fontsize=12, pad=15
29     )
30
31     plt.tight_layout()
32     plt.savefig(output_img, dpi=150, bbox_inches='tight')
33     print(f"Saved: {output_img}")
34     plt.close()
35
36 if __name__ == "__main__":
37     if len(sys.argv) < 4:
38         print("Usage: python ShowMatrix.py <matrix.csv> <header1> <header2>")
39         sys.exit(1)
40
41     plot_heatmap(sys.argv[1], sys.argv[2], sys.argv[3])

```

Entonces gracias a este programa podremos visualizar los Heatmaps correspondientes de las secuencias de prueba y las secuencias mitocondriales.



(a) Heatmap de las secuencias de prueba.



(b) Heatmap de las secuencias mitocondriales.

Figura 9: Heatmap de las secuencias de prueba (a) y de las secuencias mitocondriales (b).

Ambos **heatmaps** muestran el patrón característico de **Needleman-Wunsch**: un gradiente que va del amarillo claro en la esquina superior derecha al rojo oscuro en la esquina inferior derecha, lo cual es completamente esperado, ya que el algoritmo acumula el **score** progresivamente hacia la celda final $[n][m]$, que es donde siempre termina el alineamiento global.

La banda diagonal más oscura que desciende hacia esa esquina representa las zonas donde las secuencias divergen y se acumulan penalizaciones por **gaps**. Que el máximo **score** esté concentrado en esa esquina inferior derecha confirma visualmente que el algoritmo funcionó correctamente en ambos casos.

5. Programa completo

5.1. Código C++

```

1  #include <iostream>
2  #include <fstream>
3  #include <vector>
4  #include <algorithm>
5  #include <iomanip>
6  #include <chrono>
7  using namespace std;
8
9  const int LINE_WIDTH = 60;
10 const int MATCH_SCORE = 2;
11 const int MISMATCH_PENALTY = -1;
12 const int GAP_PENALTY = -2;
13
14 struct FastaSequences {
15     string header;
16     string sequence;
17 };
18
19 struct AlignmentResult {
20     string aligned_seq1;
21     string aligned_seq2;
22     int score;
23     int start_pos_seq1, end_pos_seq1;
24     int start_pos_seq2, end_pos_seq2;
25     vector<vector<int>> score_matrix;
26     vector<pair<int, int>> alignment_path;
27 };
28

```



```
29 void read_fasta(const string &filename, vector<FastaSequences> &sequences) {
30     ifstream file(filename);
31     if (!file.is_open()) {
32         cerr << "Error opening file: " << filename << endl;
33         return;
34     }
35
36     auto strip_cr = [](string &line) {
37         if (!line.empty() && line.back() == '\\r') {
38             line.pop_back();
39         }
40     };
41
42     string line, header, sequence;
43
44     while (getline(file, line)) {
45         strip_cr(line);
46         if (line.empty()) {
47             continue;
48         }
49
50         if (line[0] == '>') {
51             if (!header.empty()) {
52                 sequences.push_back({header, sequence});
53                 sequence.clear();
54             }
55             string raw_header = line.substr(1);
56             size_t first_space = raw_header.find(' ');
57             header = (first_space != string::npos) ? raw_header.substr(0,
58                 first_space) : raw_header;
59         }
60         else {
61             for (char &c : line) {
62                 c = toupper((unsigned char)c);
63             }
64             sequence += line;
65         }
66
67         if (!header.empty()) {
68             sequences.push_back({header, sequence});
69         }
70
71         file.close();
72     }
73
74     AlignmentResult needleman_wunsch(const string &sequence1, const string &sequence2,
75         int match_score, int mismatch_penalty, int gap_penalty) {
76         int n = (int)sequence1.size();
77         int m = (int)sequence2.size();
78
79         vector<vector<int>> score_matrix(n + 1, vector<int>(m + 1, 0));
80
81         for (int i = 0; i <= n; i++) {
82             score_matrix[i][0] = i * gap_penalty;
83         }
84         for (int j = 0; j <= m; j++) {
85             score_matrix[0][j] = j * gap_penalty;
86         }
87
88         for (int i = 1; i <= n; i++) {
89             for (int j = 1; j <= m; j++) {
```



```

89         int score_diagonal = score_matrix[i - 1][j - 1] + (sequence1[i - 1] ==
90             sequence2[j - 1] ? match_score : mismatch_penalty);
91         int score_up = score_matrix[i - 1][j] + gap_penalty;
92         int score_left = score_matrix[i][j - 1] + gap_penalty;
93
94         if (score_diagonal >= score_up && score_diagonal >= score_left) {
95             score_matrix[i][j] = score_diagonal;
96         }
97         else if (score_up >= score_left) {
98             score_matrix[i][j] = score_up;
99         }
100        else {
101            score_matrix[i][j] = score_left;
102        }
103    }
104
105    string aligned_sequence1, aligned_sequence2;
106    int i = n, j = m;
107
108    while (i > 0 && j > 0) {
109        int score_diagonal = score_matrix[i - 1][j - 1] + (sequence1[i - 1] ==
110            sequence2[j - 1] ? match_score : mismatch_penalty);
111        int score_up = score_matrix[i - 1][j] + gap_penalty;
112        int score_left = score_matrix[i][j - 1] + gap_penalty;
113
114        if (score_matrix[i][j] == score_diagonal) {
115            aligned_sequence1 += sequence1[i - 1];
116            aligned_sequence2 += sequence2[j - 1];
117            i--;
118            j--;
119        }
120        else if (score_matrix[i][j] == score_up) {
121            aligned_sequence1 += sequence1[i - 1];
122            aligned_sequence2 += '-';
123            i--;
124        }
125        else {
126            aligned_sequence1 += '-';
127            aligned_sequence2 += sequence2[j - 1];
128            j--;
129        }
130    }
131
132    while (i > 0) {
133        aligned_sequence1 += sequence1[i - 1];
134        aligned_sequence2 += '-';
135        i--;
136    }
137
138    while (j > 0) {
139        aligned_sequence1 += '-';
140        aligned_sequence2 += sequence2[j - 1];
141        j--;
142    }
143
144    reverse(aligned_sequence1.begin(), aligned_sequence1.end());
145    reverse(aligned_sequence2.begin(), aligned_sequence2.end());
146
147    return {aligned_sequence1, aligned_sequence2, score_matrix[n][m], 0, n - 1, 0,
148        m - 1, score_matrix, {}};

```

```
149 void save_alignment(const AlignmentResult &result, const string &header1, const
    string &header2) {
150     string base_path = "File txt/" + header1 + "_vs_" + header2 + "_alignment";
151
152     {
153         string csv_path = base_path + "_matrix.csv";
154         ofstream csv_file(csv_path);
155
156         if (!csv_file.is_open()) {
157             cerr << "Error opening file for writing: " << csv_path << endl;
158             return;
159         }
160
161         int n = (int)result.score_matrix.size();
162         int m = (int)result.score_matrix[0].size();
163         for (int i = 0; i < n; i++) {
164             for (int j = 0; j < m; j++) {
165                 csv_file << result.score_matrix[i][j];
166                 if (j < m - 1) {
167                     csv_file << ",";
168                 }
169             }
170             csv_file << "\n";
171         }
172         csv_file.close();
173         cout << "Matrix saved to: " << csv_path << endl;
174     }
175
176     {
177         string txt_path = base_path + "_alignment.txt";
178         ofstream txt_file(txt_path);
179
180         if (!txt_file.is_open()) {
181             cerr << "Error opening file for writing: " << txt_path << endl;
182             return;
183         }
184
185         txt_file << "Score: " << result.score << "\n\n";
186
187         int len = (int)result.aligned_seq1.size();
188
189         for (int i = 0; i < len; i += LINE_WIDTH) {
190             int end = min(i + LINE_WIDTH, len);
191
192             string seq1 = result.aligned_seq1.substr(i, end - i);
193             string seq2 = result.aligned_seq2.substr(i, end - i);
194
195             string match_line;
196             for (int k = 0; k < (int)seq1.size(); k++) {
197                 if (seq1[k] == seq2[k]) {
198                     match_line += '|';
199                 }
200                 else if (seq1[k] == '-' || seq2[k] == '-') {
201                     match_line += ' ';
202                 }
203                 else {
204                     match_line += '.';
205                 }
206             }
207
208             txt_file << seq1 << "\n";
209             txt_file << match_line << "\n";
210             txt_file << seq2 << "\n\n";

```

```

211     }
212
213     txt_file.close();
214     cout << "Alignment saved to: " << txt_path << endl;
215 }
216 }
217
218 int main(int argc, char *argv[]) {
219     if (argc != 3) {
220         cerr << "Usage: " << argv[0] << " <file1.fasta> <file2.fasta>\n";
221         return 1;
222     }
223
224     vector<FastaSequences> sequences1, sequences2;
225     read_fasta(argv[1], sequences1);
226     read_fasta(argv[2], sequences2);
227
228     if (sequences1.empty() || sequences2.empty()) {
229         cerr << "Error opening files.\n";
230         return 1;
231     }
232
233     auto start = chrono::high_resolution_clock::now();
234     AlignmentResult result = needleman_wunsch(
235         sequences1[0].sequence,
236         sequences2[0].sequence,
237         MATCH_SCORE, MISMATCH_PENALTY, GAP_PENALTY
238     );
239     auto end = chrono::high_resolution_clock::now();
240     chrono::duration<double> elapsed = end - start;
241     cout << "Time: " << fixed << setprecision(3) << elapsed.count() << " seconds\n"
242         ;
243
244     save_alignment(result, sequences1[0].header, sequences2[0].header);
245
246     return 0;
247 }

```

5.2. Código Python

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import sys
4 import os
5
6 def load_matrix(matrix_path):
7     return np.loadtxt(matrix_path, delimiter=",", dtype=int)
8
9 def plot_heatmap(matrix_path, header1, header2):
10    print(f"Reading matrix: {matrix_path}")
11
12    H = load_matrix(matrix_path)
13    print(f"Matrix size: {H.shape}")
14
15    output_img = os.path.join("Heatmaps", header1 + "_vs_" + header2 + "
16        _alignment_matrix.png")
17
18    fig, ax = plt.subplots(figsize=(14, 12))
19
20    im = ax.imshow(H, cmap="YlOrRd", aspect="auto", interpolation="nearest")
21    plt.colorbar(im, ax=ax, label="Score", fraction=0.03, pad=0.04)

```

```
22     ax.set_xlabel("Sequence 2 (positions)", fontsize=11)
23     ax.set_ylabel("Sequence 1 (positions)", fontsize=11)
24     ax.set_title(
25         f"Needleman-Wunsch Score Matrix\nSize={H.shape} | Max={H.max()} | Min={H.
26             min()} ",
27         fontsize=12, pad=15
28     )
29
30     plt.tight_layout()
31     plt.savefig(output_img, dpi=150, bbox_inches='tight')
32     print(f"Saved: {output_img}")
33     plt.close()
34
35 if __name__ == "__main__":
36     if len(sys.argv) < 4:
37         print("Usage: python ShowMatrix.py <matrix.csv> <header1> <header2>")
38         sys.exit(1)
39
40     plot_heatmap(sys.argv[1], sys.argv[2], sys.argv[3])
```